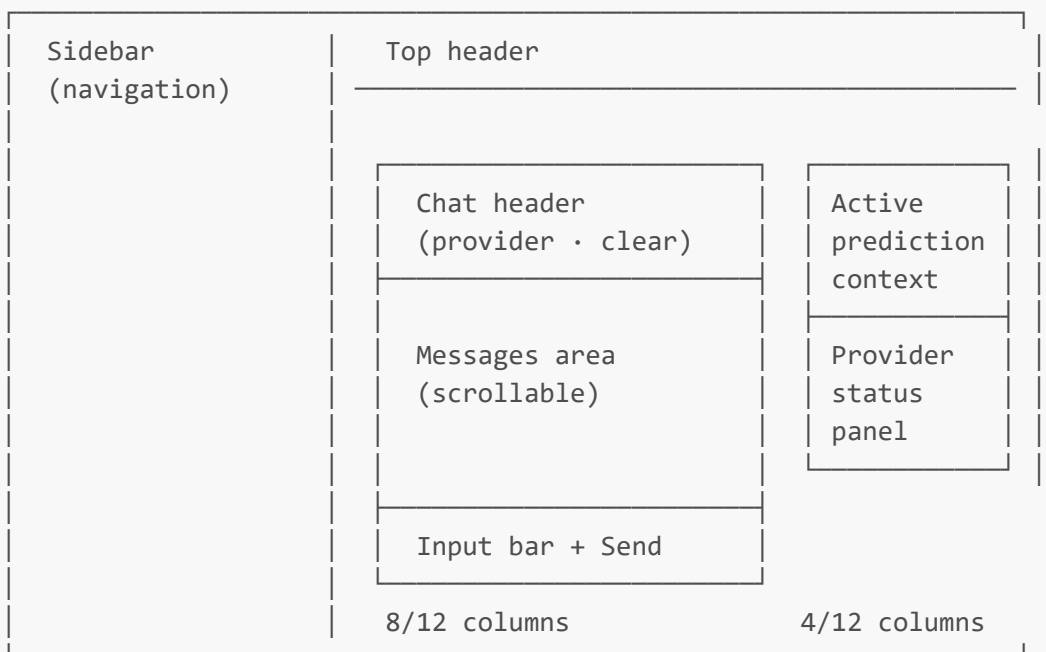


Copilot Page — Complete Component Guide

A deep-dive into every panel and control on the **Copilot** page of GenomelQ. For each component this document covers:

-  **What it is** (plain-language description)
-  **What it tells you** (how to read the output)
-  **How it works under the hood** (the algorithm)
-  **How to use it** (the user flow)
-  **Practical tips** (when it's useful, when to be cautious)

Page Layout Overview



The Copilot page is a **12-column responsive grid**: the chat takes 8 columns on the left, and a right rail (4 columns) shows the active prediction context and provider status. On mobile both stack.

1. Chat Header (Provider Selector + Clear)

What it is

The top bar of the chat panel. Shows the bot's identity, knowledge-base size, the active LLM provider, and a **Clear** button.

What it tells you

Three pieces of information:

Element	Meaning
GenomeIQ Copilot title	The chatbot's name.
Subtitle (e.g. 29 facts indexed • gemini mode)	How many curated facts are in the knowledge base, and which provider is currently selected.
Provider dropdown	Switch between DEMO, GEMINI, OLLAMA on the fly — no restart needed.
Clear button	Wipes the chat history.

⚙️ How it works under the hood

The provider list comes from `GET /chat/status` (in `src/rag/chat.py` → `status()`). Each option appears with an availability flag:

- **DEMO** is always `available: true` (template-based, offline).
- **GEMINI** is `available: true` only if `GEMINI_API_KEY` is set in `.env` and the `google-genai` package is installed.
- **OLLAMA** is `available: true` only if a local Ollama server is running at the configured `OLLAMA_BASE_URL` (default `http://localhost:11434`).

Unavailable options appear in the dropdown but are disabled, with a `(unavailable)` suffix.

📖 How to use it

1. **Select a provider** by clicking the dropdown.
2. Send messages — every new message uses the currently-selected provider.
3. Click **Clear** to start a fresh conversation.

💡 Practical tips

- You can switch providers **mid-conversation** to compare answer styles. Demo's responses are short and templated; Gemini's are conversational; Ollama's depend on the local model.
- If the dropdown shows GEMINI as `(unavailable)`, your `.env` is probably missing the key. See `HOW_TO_RUN_IN_VSCODE.md` section 4.
- Switching providers does **not** clear the chat history — the new provider sees the old messages as context.

2. 🚀 Empty State (Suggestion Buttons)

🎯 What it is

When the chat is empty, the messages area shows an introduction icon, a call-to-action, and **5 suggestion buttons** to bootstrap the conversation.

💬 What it tells you

The 5 default suggestions:

1. "What is BRCA1 and how does it relate to cancer?"
2. "Explain the role of APP and PSEN1 in Alzheimer's disease."
3. "What does the OOD detector mean by high-risk?"
4. "Why might my Diabetes prediction have low confidence?"
5. "How does GenomelQ's classifier work?"

These cover the four main topic categories: **gene-level** questions, **disease overviews**, **classifier internals**, and **prediction interpretation**.

⚙️ How it works under the hood

Defined as a JS array in `frontend/static/app.js`:

```
copilot: {  
  suggestions: [  
    "What is BRCA1 and how does it relate to cancer?",  
    "Explain the role of APP and PSEN1 in Alzheimer's disease.",  
    ...  
  ]  
}
```

When clicked, the suggestion text is passed straight to `sendCopilotMessage(text)` — same flow as a manually-typed message.

📖 How to use it

Just click any suggestion to fire it as your first question.

💡 Practical tips

- The empty state is also a **discoverability tool** — it tells a first-time user what kinds of questions the Copilot can answer.
- Want to change the suggestions? Edit the `copilot.suggestions` array in `app.js` and reload.

3. 📄 Messages Area (Conversation Thread)

🎯 What it is

The scrollable middle section that displays every user/assistant message in sequence.

💬 What it tells you

Each message bubble has:

- **Role-based positioning**: user messages are right-aligned (brand color), assistant messages are left-aligned (neutral).
- **Markdown rendering**: bold, italic, inline code, line breaks all render correctly.

- **Source chips** (assistant only): small tags showing which knowledge-base facts were retrieved for that answer.
- **Notes** (assistant only): optional warnings or hints from the provider (e.g., "Gemini quota exhausted").

⚙️ How it works under the hood

The state lives in `Alpine.data().copilot.messages`, an array of `{ role, content, retrieved, provider, note }` objects. Each new message triggers reactivity → Alpine re-renders the list, then `scrollCopilotToBottom()` smoothly scrolls to the bottom.

Markdown is rendered safely by `formatMarkdown(text)` in `app.js` — escapes HTML first, then applies a small set of regex transformations (`bold`, `italic`, ``code``, line breaks).

📖 How to use it

- **Scroll up** to read older messages — the auto-scroll only kicks in when a new message arrives.
- **Hover** over a source chip to see its full content (the chip's `title` attribute reveals the fact text).

💡 Practical tips

- The chat **persists only for the current session** (Alpine state, not localStorage). Refreshing the page wipes it.
- If you want a permanent transcript, copy-paste before refreshing.

4. 🔍 Source Chips (Cited Facts)

🎯 What it is

Below every assistant message, a row of small chips identifying the **knowledge-base facts** the Copilot used to ground its answer.

💬 What it tells you

Each chip shows:

- **Title** of the fact (e.g. `BRCA1 – breast cancer 1, early onset`)
- **Similarity score** (• 79%) — how relevant this fact was to your question

The full fact text is in the chip's `title` tooltip — hover with your mouse to read it.

⚙️ How it works under the hood

The retrieval flow:

1. User question → embedded by `sentence-transformers/all-MiniLM-L6-v2` → 384-dim vector
2. **FAISS** searches `models/rag_index.faiss` for the top-K nearest facts
3. Top-K facts are passed to the LLM provider as context
4. The same top-K is returned in the JSON response so the UI can show them as chips

Code path: `src/rag/retriever.py` → `search()`.

How to use it

Use them as **citation/provenance**: an answer with high-similarity sources from the right topic area is far more trustworthy than one with low scores or off-topic chips.

Practical tips

- A chip with **score below 30%** means the retriever isn't very confident the fact is relevant. Treat the answer with extra caution.
- Chips with **score > 70%** are typically very on-topic.
- If all chips are unrelated to your question, the knowledge base genuinely lacks coverage — see DOCS.md for how to extend it.

5. Input Bar (Send Box)

What it is

The text input + Send button at the bottom of the chat panel.

What it tells you

Element	Meaning
Text input	Where you type your question. Disabled while a response is loading.
Send button	Submits the message. Disabled while empty or while a response is loading.
Error toast (above the input)	Shows any backend error in a rose-colored box.

How it works under the hood

The input is bound to `copilot.input` via Alpine `x-model`. Two ways to send:

1. **Enter key** → triggers `sendCopilotMessage()`.
2. **Click Send** → same handler.

`sendCopilotMessage()` then:

1. Pushes a user message onto `copilot.messages`.
2. Sets `copilot.loading = true` (which shows the typing-dots animation).
3. Sends `POST /chat` with the message, provider choice, last 10 messages of history, and `prediction_context` (if you've analyzed something).
4. On response, pushes the assistant message and toggles loading off.

How to use it

1. Type your question.
2. Press **Enter** or click **Send**.
3. Wait for the typing dots to finish (Demo: instant; Gemini: 1–5 sec; Ollama: depends on local model).

Practical tips

- **Multi-line questions** can be sent — but Enter sends immediately. To insert a newline, use **Shift+Enter** (default browser behaviour).
 - If a Send fails, the error appears above the input. Common causes: network drop, Gemini quota, or Ollama not running.
-

6. Typing Indicator

What it is

Three pulsing dots that appear in the messages area while the assistant is generating a response.

What it tells you

The Copilot is busy — it has received your question and is either retrieving facts or calling the LLM.

How it works under the hood

A simple Tailwind/CSS animation defined in `frontend/static/styles.css` as `@keyframes softPulse`. The dots appear when `copilot.loading === true` and disappear when the response arrives.

Practical tips

- If the dots **never disappear**, check the browser dev-tools network tab. The most likely culprits are Gemini quota or Ollama not running. The error toast will appear once the request fails.
-

7. Active Prediction Context (Right Rail Card)

What it is

A small card on the right rail that shows your **most recent Analyze result** — and indicates that the Copilot is automatically using it as context.

What it tells you

If you've analyzed a sequence on the Analyze page, this card displays:

- The predicted disease class (color-coded)
- The confidence percentage
- The active model backbone (e.g. `model TFIDF`)
- A note: *"Copilot will weave this prediction into its answers."*

If you **haven't analyzed anything yet**, this card is hidden.

How it works under the hood

When a `/predict` call succeeds, the result is stored in `analyze.result` (Alpine state). The Copilot reads it before sending each `/chat` request:

```
const predictionContext = this.analyze.result
  ? {
    predicted_class: this.analyze.result.prediction.predicted_class,
    confidence: this.analyze.result.prediction.confidence,
    probabilities: this.analyze.result.prediction.probabilities,
    model: this.analyze.result.prediction.model,
  }
  : null;
```

The `prediction_context` field is included in the `POST /chat` body. The backend (`src/rag/providers.py`) embeds it into the LLM prompt so the model can answer questions like "Why was this Cancer?" with specific reference to your prediction.

How to use it

1. First, run any prediction on the **Analyze** page.
2. Switch to **Copilot**.
3. Notice the green prediction context card appears.
4. Ask: "Why is my prediction Cancer?" or "What does this confidence mean?"

Practical tips

- If you re-analyze a different sequence, this card updates automatically — refresh isn't needed.
- The Copilot won't invent context — if you haven't analyzed anything, it will give a generic answer.

8. Provider Status Panel (Right Rail)

What it is

A card on the right rail showing the **availability** of each LLM provider with troubleshooting hints.

What it tells you

For each provider (DEMO, GEMINI, OLLAMA):

-  Green dot = available right now
-  Gray dot = unavailable
- A short description (e.g., "Google Gemini (free tier, 1500 req/day)")
- A **Needs:** line for unavailable providers, listing exactly what's missing — e.g., "`GEMINI_API_KEY` env var" or "Install Ollama from ollama.com"

At the bottom is a tip: "set the `GEMINI_API_KEY` environment variable for the best free experience."

How it works under the hood

`src/rag/providers.py` → `list_provider_status()` runs three checks:

1. **demo** — always `True`.
2. **gemini** — `True` only if both `google.genai` is importable AND `GEMINI_API_KEY` env var is set.

3. **ollama** — pings `{base_url}/api/tags` with a 2-second timeout; **True** if HTTP 200.

Returned by **GET** `/chat/status`, polled once at page load.

How to use it

This panel is **diagnostic** — when DEMO works but GEMINI doesn't, this card tells you exactly what to fix.

Practical tips

- The status is **fetched only at page load**. If you start Ollama or set the API key, refresh the page to see the change.
- If multiple providers are available, just use the **header dropdown** to switch.

9. Clear Button

What it is

A small "Clear" button in the chat header.

What it tells you

Empties the entire conversation. The empty state with the suggestion buttons reappears.

How it works under the hood

```
clearCopilot() {
  this.copilot.messages = [];
  this.copilot.error = "";
}
```

Practical tips

- The Clear button does **not** reset the provider selection or the prediction context — only the message list.
- It does **not** clear the chat from any provider's server-side memory either; each `/chat` request is stateless on the backend (only the last 10 client-side messages are sent as history).

The Full Copilot Flow (End-to-End)

```
User types message and presses Enter
  ↓
sendCopilotMessage(message)
  ↓
[POST /chat] with { message, provider, history, prediction_context }
  ↓
Backend: src/rag/chat.py → chat()
```




Total time: **<100 ms** for Demo, **1–5 sec** for Gemini, depends on hardware for Ollama.

Cheat Sheet — Picking the Right Provider

Scenario	Best provider
Quick offline test, no setup	DEMO
Free, polished natural-language answers	GEMINI (1500 req/day)
Air-gapped / privacy-sensitive deployment	OLLAMA
Demo to an audience without internet	OLLAMA (or DEMO)
Comparing answer quality	All three back-to-back

Related Documents

- [README.md](#) — high-level project overview
- [DOCS.md](#) — comprehensive technical documentation
- [HOW_TO_RUN_IN_VSCODE.md](#) — VS Code setup and `.env` configuration
- [TESTING_GUIDE.md](#) — full test plan for the Copilot endpoints
- [Guide/ANALYZE_PAGE_GUIDE.pdf](#) — the equivalent walkthrough for the Analyze page

Part of the GenomelQ per-page documentation series.